1.项目概述

1.1.项目演示

1. 运行 “饿了么项目（Spring Cloud版）”，演示应用程序效果，演示 “点餐业务线” 整体流程。

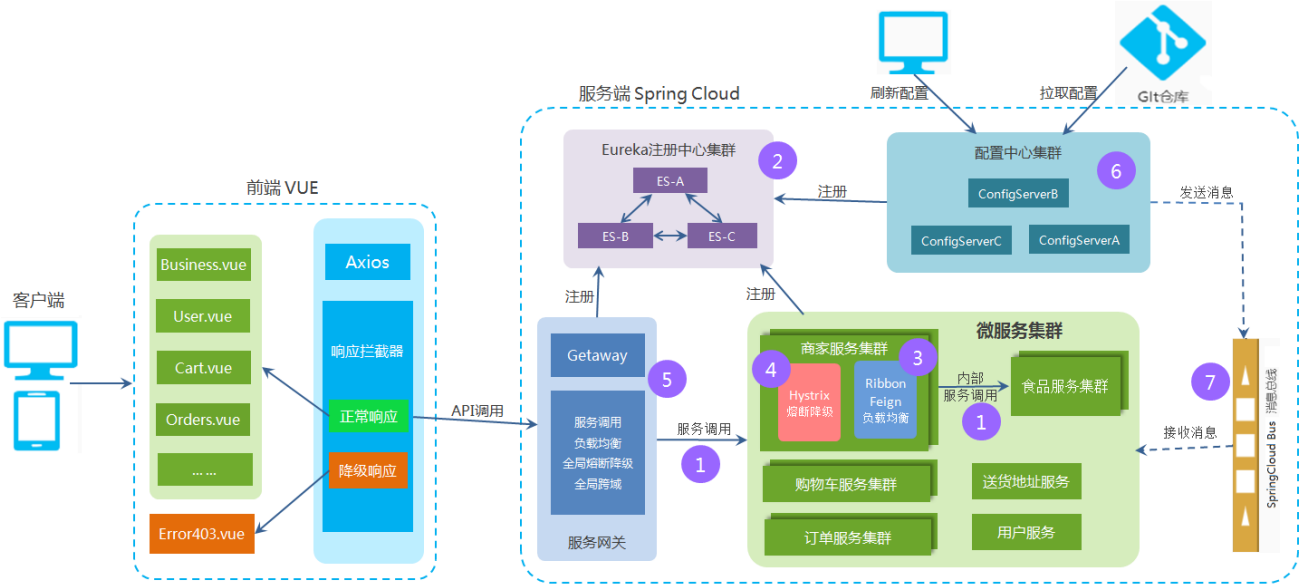
1.2.项目目标

1. 本项目主要是在“饿了么项目（SpringBoot版）”的基础上，升级为 Spring Cloud 微服务版。
2. 本项目完成后，学员将能够使用 Spring Cloud 技术实现分布式微服务系统基础设施的开发。

1.3.项目学习路线图

对于初学者来说，建议要遵循以下路线图来学习：

1.4.项目架构图



- 分阶段实施的说明：
  1. 第一阶段：只完成简单的微服务拆分与调用。
  2. 第二阶段：添加Eureka注册中心服务，以及搭建Eureka高可用服务集群。
  3. 第三阶段：添加基于Feign的内部服务调用，以及负载均衡。
  4. 第四阶段：添加服务消费者的Hystrix熔断降级处理。
  5. 第五阶段：添加Gateway微服务网关，并实现Gateway的熔断降级、负载均衡、跨域配置等处理。
  6. 第六阶段：添加Config集中配置管理，以及搭建Config高可用服务集群。
  7. 第七阶段：添加Bus配置刷新。

1.5.微服务拆分

如何进行微服务的拆分，目前并没有一个明确的标准可以参考。即便是有一些理论（比如：AKF拆分原则），也仅仅是理论上的。所以这里结合一些查阅的资料，总结一下微服务拆分的原则：

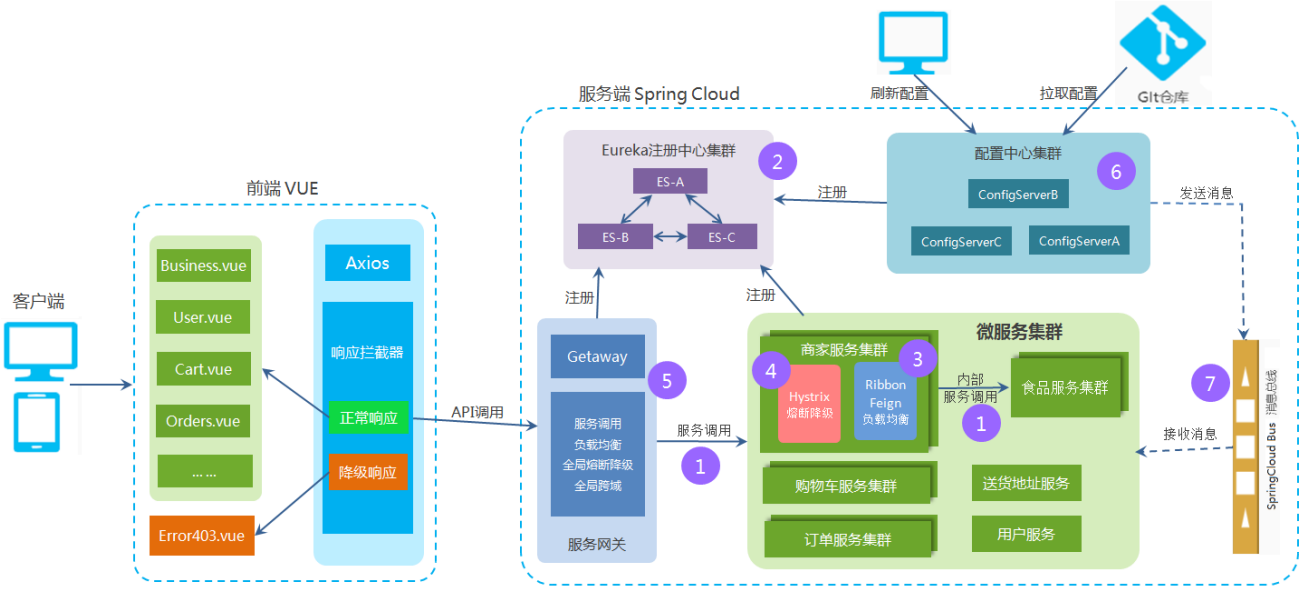
1. 单一职责，高内聚低耦合。（简单的来说：一张表一个微服务）
2. 服务粒度适中。（粒度不要太细，一个事务一个微服务）
3. 以业务模型切入。（用户模块、产品模块、订单模块，都可以成为一个微服务）
4. 考虑团队结构。（有一种理论认为：微服务的本质是团队分工问题，所以有几个团队，就拆分几个服务）
5. 渐进式拆分。（刚开始不要划分太细，可以随着迭代过程逐步优化）
6. 避免环形依赖与双向依赖。（服务拆分最多三层，两次单向调用）

据此，本项目微服务拆分如下：

- 六个微服务：商家、食品、购物车、订单、送货地址、用户。
- 商家、食品、购物车、订单：这四个微服务搭建集群，保证高可用。
- 送货地址、用户：不搭建集群。
- 商家微服务与食品微服务为内部调用关系。

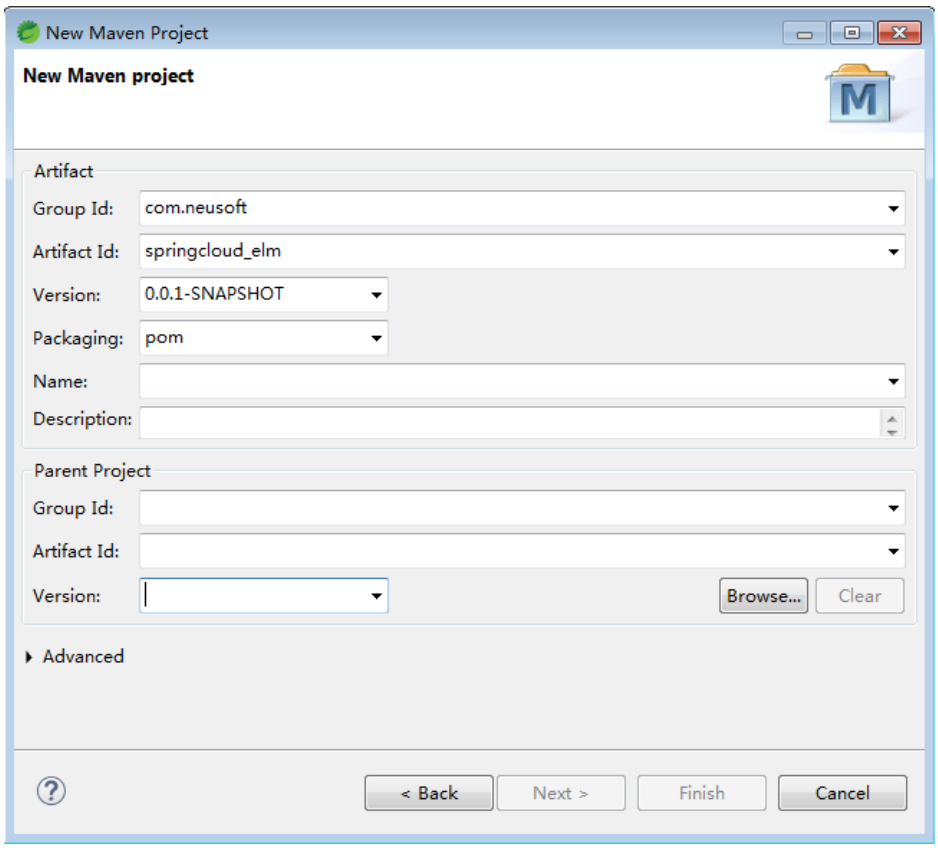
2.搭建Spring Cloud工程

本阶段项目完成总架构图中的第一阶段，实现最简单的微服务拆分与调用。



2.1.创建elm父工程

创建 Maven Project 父工程（工程名：springcloud_demo；Packaging：pom）



删除src目录，并修改pom.xml文件

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
```

```
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.neusoft</groupId>
    <artifactId>spring_cloud_demo</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <packaging>pom</packaging>

    <!-- 统一JAR包和版本号的管理 -->
    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.source>1.8</maven.compiler.source>
        <maven.compiler.target>1.8</maven.compiler.target>
    </properties>

    <!--
        dependmanagement是用在父工程中声明依赖，不引用。子模块继承之后再引包。
             作用是，锁定版本，子模块不用写grouid and version
    -->
    <dependencyManagement>

        <dependencies>
            <!--boot 2.3.3 -->
            <dependency>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-dependencies</artifactId>
                <version>2.3.3.RELEASE</version>
                <type>pom</type>
                <!-- Maven也是单继承。这里继承了spring-boot中的dependencyManagement
                     但是下面还要继承spring-cloud中的dependencyManagement。
                    使用scope=import就可以实现多继承了
                -->
                <scope>import</scope>
            </dependency>

            <!--cloud hoxton.sr9 -->
            <dependency>
                <groupId>org.springframework.cloud</groupId>
                <artifactId>spring-cloud-dependencies</artifactId>
                <version>Hoxton.SR9</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>

        </dependencies>

    </dependencyManagement>

    <!-- 创建完子工程后，此处自动添加子工程模块 -->
    <!--
    <modules>

        <module>...</module>
```

```
    </modules>
    -->

</project>
```

> 注意：根据Spring官方提供的SpringCloud与SpringBoot版本兼容说明中，SpringCloud的Hoxton版本必须使用SpringBoot的2.2.x或2.3.x版本。

2.2.创建用户微服务

在父工程下，创建 Maven Module 子工程（工程名：user_server_10100；Packaging：jar）

2.2.1.修改pom.xml文件

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>com.neusoft</groupId>
        <artifactId>springcloud_elm</artifactId>
        <version>0.0.1-SNAPSHOT</version>
    </parent>
    <artifactId>user_server_10100</artifactId>

    <dependencies>
        <!--以下是boot -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <!--热部署 gav -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>

        <!--数据层相关依赖 -->
        <dependency>
            <groupId>org.mybatis.spring.boot</groupId>
            <artifactId>mybatis-spring-boot-starter</artifactId>
            <version>2.0.1</version>
        </dependency>
```

```
        <dependency>
            <groupId>mysql</groupId>
            <artifactId>mysql-connector-java</artifactId>
            <version>5.1.6</version>
            <scope>runtime</scope>
        </dependency>
    </dependencies>
</project>
```

2.2.2.创建主启动类

```
package com.neusoft;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class MyApplication {

    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

2.2.3.创建共通返回数据

在po包下创建CommonResult类，作为共通数据格式响应给前端。

```
package com.neusoft.po;

import java.io.Serializable;

public class CommonResult<T> implements Serializable{

    private Integer code;
    private String message;
    private T result;

    public CommonResult() {}

    public CommonResult(Integer code,String message,T result) {
        this.code = code;
        this.message = message;
        this.result = result;
    }


    public Integer getCode() {
```

```
        return code;
    }
    public void setCode(Integer code) {
        this.code = code;
    }
    public String getMessage() {
        return message;
    }
    public void setMessage(String message) {
        this.message = message;
    }
    public T getResult() {
        return result;
    }
    public void setResult(T result) {
        this.result = result;
    }
}
```

2.2.4.创建Controller控制器

```
package com.neusoft.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.neusoft.po.CommonResult;
import com.neusoft.po.User;
import com.neusoft.service.UserService;

@CrossOrigin("*")   //跨域处理
@RestController
@RequestMapping("/UserController")
public class UserController {

    @Autowired
    private UserService userService;

    @GetMapping("/getUserByIdByPass/{userId}/{password}")
    public CommonResult<User> getUserByIdByPass(
            @PathVariable("userId") String userId,
            @PathVariable("password") String password) throws Exception{
        User param = new User();
        param.setUserId(userId);

        param.setPassword(password);
```

```
        User user = userService.getUserByIdByPass(param);
        return new CommonResult(200,"success",user);
    }

    @GetMapping("/getUserById/{userId}")
    public CommonResult<Integer> getUserById(@PathVariable("userId") String userId) throws
Exception{
        int result = userService.getUserById(userId);
        return new CommonResult(200,"success",result);
    }

    @PostMapping("/saveUser/{userId}/{password}/{userName}/{userSex}")
    public CommonResult<Integer> saveUser(
                @PathVariable("userId") String userId,
                @PathVariable("password") String password,
                @PathVariable("userName") String userName,
                @PathVariable("userSex") Integer userSex) throws Exception{
        User param = new User();
        param.setUserId(userId);
        param.setPassword(password);
        param.setUserName(userName);
        param.setUserSex(userSex);
        int result = userService.saveUser(param);
        return new CommonResult(200,"success",result);
    }
}
```

2.2.5.创建service、mapper等组件

在“饿了么项目（SpringBoot版）”中有详细代码，这里不再累述。

2.2.6.创建application.yml配置文件

```
server:
    port: 10100

logging:
    level:
        org.springframework: debug
        com.neusoft.mapper: debug

spring:
    application:
        name: user-server
    datasource:
        username: root
        password: 123

        url: jdbc:mysql://localhost:3306/elm?characterEncoding=utf-8
```

```
        driver-class-name: com.mysql.jdbc.Driver

mybatis:
    mapper-locations: classpath:mapper/*.xml
    type-aliases-package: com.neusoft.po
```

2.2.7.启动并测试

浏览器地址栏输入：http://localhost:10100/user/getUserByIdByPass/12345671111/123 测试是否能正确返回数据。

2.2.8.修改Vue前端

Vue前端工程修改一下几处：

1. 删除 main.js 文件中的：axios.defaults.baseURL = 'http://localhost:8080/'; 因为不同微服务的IP和端口都不一致。
2. 使用axios请求时，一律修改成 restful 形式：

```
//登录请求
let url =
`http://localhost:10100/UserController/getUserByIdByPass/${this.userId}/${this.password}`;
//遵循rest规范，所以查询使用get
this.$axios.get(url).then(response=>{
    //这里注意写法的改变：response.data.result
    let user = response.data.result;

    if(user==null||user==''){
        alert('用户名或密码不正确！');
    }else{
        //sessionstorage有容量限制，为了防止数据溢出，所以不将userImg数据放入session中
        user.userImg = '';
        this.$setSessionStorage('user',user);
        this.$router.go(-1);
    }
}).catch(error=>{
    console.error(error);
});
```

2.3.创建食品微服务

在父工程下，创建 Maven Module 子工程（工程名：food_server_10200；Packaging：jar）

食品微服务工程中的pom.xml文件、主启动类、CommonResult类等，与用户微服务一致，这里不再累述。

2.3.1.创建Controller控制器

```
@CrossOrigin("*")
@RestController
@RequestMapping("/FoodController")
public class FoodController {

    @Autowired
    private FoodService foodService;

    @GetMapping("/listFoodByBusinessId/{businessId}")
    public CommonResult<List> listFoodByBusinessId(@PathVariable("businessId") Integer
businessId) throws Exception{
        List<Food> list = foodService.listFoodByBusinessId(businessId);
        return new CommonResult(200,"success",list);
    }
}
```

2.3.2.创建application.yml配置文件

```
server:
    port: 10200

logging:
    level:
        org.springframework: debug
        com.neusoft.mapper: debug

spring:
    application:
        name: food-server
    datasource:
        username: root
        password: 123
        url: jdbc:mysql://localhost:3306/elm?characterEncoding=utf-8
        driver-class-name: com.mysql.jdbc.Driver

mybatis:
    mapper-locations: classpath:mapper/*.xml
    type-aliases-package: com.neusoft.po
```

2.4.创建商家微服务

在父工程下，创建 Maven Module 子工程（工程名：business_server_10300；Packaging：jar）

2.4.1.创建主启动类

```
@SpringBootApplication
public class MyApplication {
    //向容器中添加RestTemplate实例
    @Bean
    public RestTemplate getRestTemplate() {
        return new RestTemplate();
    }

    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

2.4.2.创建Controller控制器

```
@CrossOrigin("*")
@RestController
@RequestMapping("/BusinessController")
public class BusinessController {

    @Autowired
    private BusinessService businessService;
    @Autowired
    private RestTemplate restTemplate;

    @GetMapping("/listBusinessByOrderTypeId/{orderTypeId}")
    public CommonResult<List> listBusinessByOrderTypeId(@PathVariable("orderTypeId") Integer
orderTypeId) throws Exception{
        List<Business> list = businessService.listBusinessByOrderTypeId(orderTypeId);
        return new CommonResult(200,"success",list);
    }

    @GetMapping("/getBusinessById/{businessId}")
    public CommonResult<Business> getBusinessById(@PathVariable("businessId") Integer
businessId) throws Exception{
        Business business = businessService.getBusinessById(businessId);
        //在商家微服务中调用食品微服务
        CommonResult<List> result =
restTemplate.getForObject("http://localhost:10200/FoodController/listFoodByBusinessId/"+business
Id, CommonResult.class);
        if(result.getCode()==200) {
            business.setFoodList(result.getResult());
        }
        return new CommonResult(200,"success",business);
    }
}
```

> 服务消费者调用服务提供者时，采用的是：主机地址与端口使用硬编码形式。等到第2阶段项目时，会配置Eureka服务注册中心，这里就可以不硬编码了，而是从Eureka服务中获取主机地址与端口。

2.4.3.创建application.yml配置文件

```yaml
server:
    port: 10300

spring:
    application:
        name: business-server
    #业务配置
    datasource:
        username: root
        password: 123
        url: jdbc:mysql://localhost:3306/elm?characterEncoding=utf-8
        driver-class-name: com.mysql.jdbc.Driver

#业务配置
logging:
    level:
        org.springframework: debug
        com.neusoft.mapper: debug
#业务配置
mybatis:
    mapper-locations: classpath:mapper/*.xml
    type-aliases-package: com.neusoft.po
```

2.5.创建购物车微服务

在父工程下，创建 Maven Module 子工程（工程名：cart_server_10400；Packaging：jar）

2.5.1.创建Controller控制器

```java
@CrossOrigin("*")
@RestController
@RequestMapping("/CartController")
public class CartController {

    @Autowired
    private CartService cartService;

    @GetMapping("/listCart/{userId}")
    public CommonResult<List> listCart(@PathVariable("userId") String userId) throws Exception{

        Cart param = new Cart();
```

```
        param.setUserId(userId);
        List<Cart> list = cartService.listCart(param);
        return new CommonResult(200,"success",list);
    }

    @GetMapping("/listCart/{userId}/{businessId}")
    public CommonResult<List> listCart(
            @PathVariable("userId") String userId,
            @PathVariable("businessId") Integer businessId) throws Exception{
        Cart param = new Cart();
        param.setUserId(userId);
        param.setBusinessId(businessId);
        List<Cart> list = cartService.listCart(param);
        return new CommonResult(200,"success",list);
    }

    @PostMapping("/saveCart/{userId}/{businessId}/{foodId}")
    public CommonResult<Integer> saveCart(
            @PathVariable("userId") String userId,
            @PathVariable("businessId") Integer businessId,
            @PathVariable("foodId") Integer foodId) throws Exception{
        Cart param = new Cart();
        param.setUserId(userId);
        param.setBusinessId(businessId);
        param.setFoodId(foodId);
        int result = cartService.saveCart(param);
        return new CommonResult(200,"success",result);
    }

    @PutMapping("/updateCart/{userId}/{businessId}/{foodId}/{quantity}")
    public CommonResult<Integer> updateCart(
            @PathVariable("userId") String userId,
            @PathVariable("businessId") Integer businessId,
            @PathVariable("foodId") Integer foodId,
            @PathVariable("quantity") Integer quantity) throws Exception{
        Cart param = new Cart();
        param.setUserId(userId);
        param.setBusinessId(businessId);
        param.setFoodId(foodId);
        param.setQuantity(quantity);
        int result = cartService.updateCart(param);
        return new CommonResult(200,"success",result);
    }

    @DeleteMapping("/removeCart/{userId}/{businessId}/{foodId}")
    public CommonResult<Integer> removeCart(
            @PathVariable("userId") String userId,
            @PathVariable("businessId") Integer businessId,
            @PathVariable("foodId") Integer foodId) throws Exception{
        Cart param = new Cart();
        param.setUserId(userId);
        param.setBusinessId(businessId);

        param.setFoodId(foodId);
```

```
        int result = cartService.removeCart(param);
        return new CommonResult(200,"success",result);
    }
}
```

2.5.2.创建application.yml配置文件

```yml
server:
    port: 10400

spring:
    application:
        name: cart-server
    #业务配置
    datasource:
        username: root
        password: 123
        url: jdbc:mysql://localhost:3306/elm?characterEncoding=utf-8
        driver-class-name: com.mysql.jdbc.Driver

#业务配置
logging:
    level:
        org.springframework: debug
        com.neusoft.mapper: debug
#业务配置
mybatis:
    mapper-locations: classpath:mapper/*.xml
    type-aliases-package: com.neusoft.po
```

2.6.创建送货地址微服务

在父工程下，创建 Maven Module 子工程（工程名：deliveryaddress_server_10500；Packaging：jar）

2.6.1.创建Controller控制器

```
@CrossOrigin("*")
@RestController
@RequestMapping("/DeliveryAddressController")
public class DeliveryAddressController {

    @Autowired
    private DeliveryAddressService deliveryAddressService;

    @GetMapping("/listDeliveryAddressByUserId/{userId}")
```

```
    public CommonResult<List> listDeliveryAddressByUserId(@PathVariable("userId") String userId)
throws Exception{
        List<DeliveryAddress> list = deliveryAddressService.listDeliveryAddressByUserId(userId);
        return new CommonResult(200,"success",list);
    }

    @GetMapping("/getDeliveryAddressById/{daId}")
    public CommonResult<DeliveryAddress> getDeliveryAddressById(@PathVariable("daId") Integer
daId) throws Exception{
        DeliveryAddress deliveryAddress = deliveryAddressService.getDeliveryAddressById(daId);
        return new CommonResult(200,"success",deliveryAddress);
    }


@PostMapping("/saveDeliveryAddress/{contactName}/{contactSex}/{contactTel}/{address}/{userId}")
    public CommonResult<Integer> saveDeliveryAddress(
                @PathVariable("contactName") String contactName,
                @PathVariable("contactSex") Integer contactSex,
                @PathVariable("contactTel") String contactTel,
                @PathVariable("address") String address,
                @PathVariable("userId") String userId) throws Exception{
        DeliveryAddress param = new DeliveryAddress();
        param.setContactName(contactName);
        param.setContactSex(contactSex);
        param.setContactTel(contactTel);
        param.setAddress(address);
        param.setUserId(userId);
        int result = deliveryAddressService.saveDeliveryAddress(param);
        return new CommonResult(200,"success",result);
    }


@PutMapping("/updateDeliveryAddress/{daId}/{contactName}/{contactSex}/{contactTel}/{address}")
    public CommonResult<Integer> updateDeliveryAddress(
                @PathVariable("daId") Integer daId,
                @PathVariable("contactName") String contactName,
                @PathVariable("contactSex") Integer contactSex,
                @PathVariable("contactTel") String contactTel,
                @PathVariable("address") String address) throws Exception{
        DeliveryAddress param = new DeliveryAddress();
        param.setDaId(daId);
        param.setContactName(contactName);
        param.setContactSex(contactSex);
        param.setContactTel(contactTel);
        param.setAddress(address);
        int result = deliveryAddressService.updateDeliveryAddress(param);
        return new CommonResult(200,"success",result);
    }

    @DeleteMapping("/removeDeliveryAddress/{daId}")
    public CommonResult<Integer> removeDeliveryAddress(@PathVariable("daId") Integer daId)
throws Exception{

        int result = deliveryAddressService.removeDeliveryAddress(daId);
```

```
        return new CommonResult(200,"success",result);
    }
}
```

2.6.2.创建application.yml配置文件

```yaml
server:
    port: 10500

spring:
    application:
        name: deliveryaddress-server
    #业务配置
    datasource:
        username: root
        password: 123
        url: jdbc:mysql://localhost:3306/elm?characterEncoding=utf-8
        driver-class-name: com.mysql.jdbc.Driver

#业务配置
logging:
    level:
        org.springframework: debug
        com.neusoft.mapper: debug
#业务配置
mybatis:
    mapper-locations: classpath:mapper/*.xml
    type-aliases-package: com.neusoft.po
```

2.7. 创建订单服务

在父工程下，创建 Maven Module 子工程（工程名：orders_server_10600；Packaging：jar）

2.7.1.创建Controller控制器

```java
@CrossOrigin("*")
@RestController
@RequestMapping("/OrdersController")
public class OrdersController {

    @Autowired
    private OrdersService ordersService;

    @PostMapping("/createOrders/{userId}/{businessId}/{daId}/{orderTotal}")
    public CommonResult<Integer> createOrders(
```

```
                @PathVariable("userId") String userId,
                @PathVariable("businessId") Integer businessId,
                @PathVariable("daId") Integer daId,
                @PathVariable("orderTotal") Double orderTotal) throws Exception{
        Orders orders = new Orders();
        orders.setUserId(userId);
        orders.setBusinessId(businessId);
        orders.setDaId(daId);
        orders.setOrderTotal(orderTotal);
        int orderId = ordersService.createOrders(orders);
        return new CommonResult(200,"success (10600) ",orderId);
    }

    @GetMapping("/getOrdersById/{orderId}")
    public CommonResult<Orders> getOrdersById(
                @PathVariable("orderId") Integer orderId) throws Exception{
        Orders orders = ordersService.getOrdersById(orderId);
        return new CommonResult(200,"success (10600) ",orders);
    }

    @GetMapping("/listOrdersByUserId/{userId}")
    public CommonResult<List> listOrdersByUserId(
                @PathVariable("userId") String userId) throws Exception{
        List<Orders> list = ordersService.listOrdersByUserId(userId);
        return new CommonResult(200,"success (10600) ",list);
    }
}
```

2.7.2.创建application.yml配置文件

```
server:
    port: 10600

spring:
    application:
        name: orders-server
    #业务配置
    datasource:
        username: root
        password: 123
        url: jdbc:mysql://localhost:3306/elm?characterEncoding=utf-8
        driver-class-name: com.mysql.jdbc.Driver

#业务配置
logging:
    level:
        org.springframework: debug
        com.neusoft.mapper: debug
#业务配置

mybatis:
```

```yaml
    mapper-locations: classpath:mapper/*.xml
    type-aliases-package: com.neusoft.po
```